LAB: GPIO Digital InOut(eval board)

LAB: GPIO Digital InOut

Date: 2026-09-21

Author/Partner: Choi Junghyun

Demo Video: https://youtu.be/oBjm3dOoZGE

PDF version:

Introduction

In this lab, you are required to create a simple program that toggles multiple LEDs using a push-button input.

You must create your own drivers for GPIO digital input and output control.

You must submit

- LAB Report (*.pdf)
- Zip source files(*main*.c, ecRCC.h, ecGPIO.h etc...).
    - Only the source files.
    - Do not submit project files

Requirement

Hardware

- MCU
    - NUCLEO-F411RE
- Eval Board
- Actuator/Sensor/Others:

Software

- PlatformIO, CMSIS, EC_HAL library

Problem 0: STM-Arduino

We are going to create a simple program that turns LED(LD2) on and off by pressing the user button(BT1), using Arduino Syntax

GPIO Digital In/Out

Create a new project under the directory `\repos\EC\lab\`

- The project folder name is "**\LAB_GPIO_DIO_LED".**

Configures the specified pin to behave either as an input or an output.

```
pinMode(pin, mode)
```

- pin: the pin number to set the model of.
- mode: INPUT, OUTPUT or INPUT_PULLUP.

> Look up for pinMode() function in arduino reference for detail description.

Example code

Open *Arduino IDE* and Create a new program named as '**TU_arduino_GPIO_LED_button.ino**'.

Write the following source code: source code.

```
const int btnPin = 3;
const int ledPin = 13;

int btnState = HIGH;

void setup() {
  pinMode(ledPin, OUTPUT);
  pinMode(btnPin, INPUT);
}

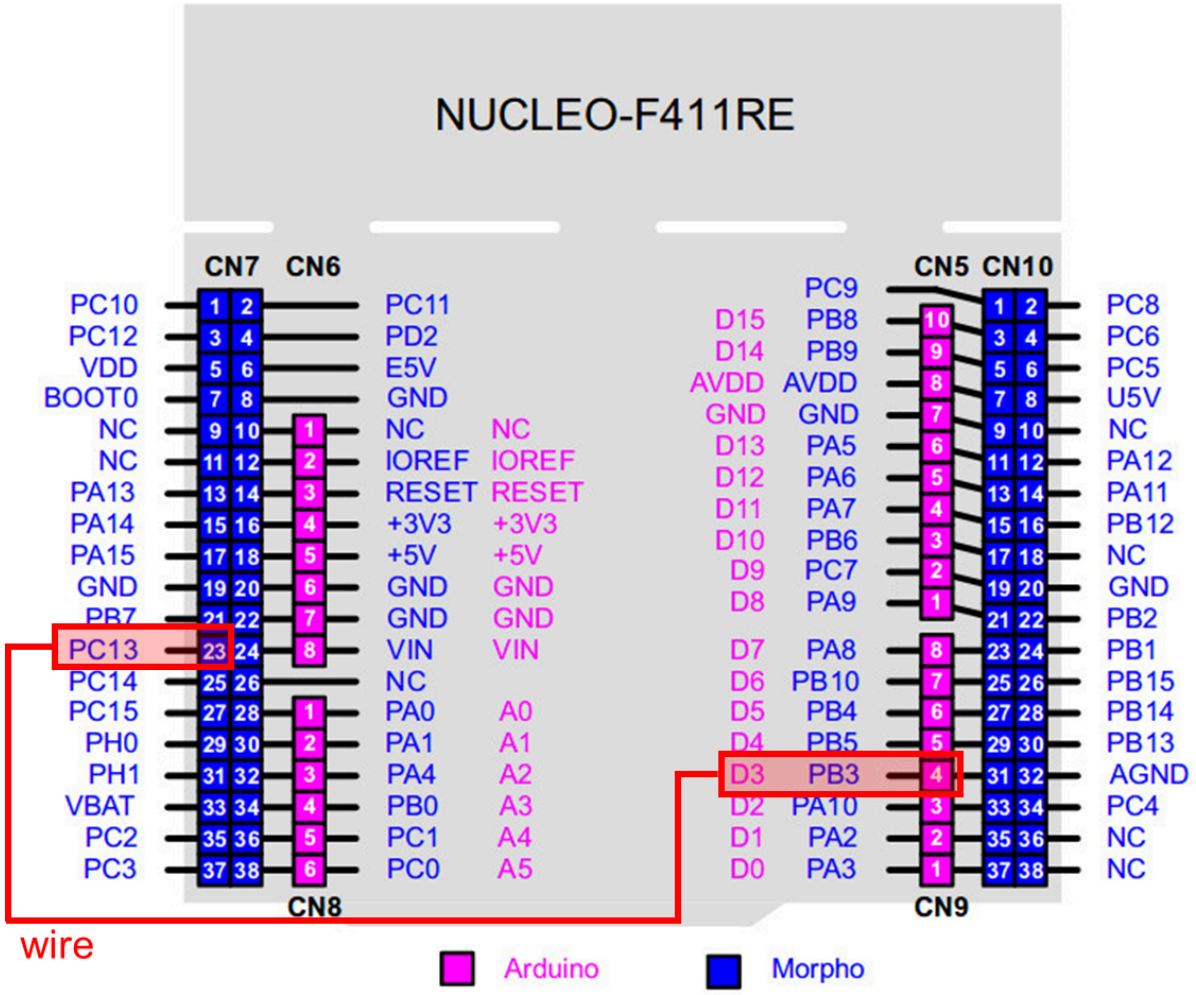
void loop() {
  btnState = digitalRead(btnPin);

  if (btnState == HIGH)
    digitalWrite(ledPin, LOW);

  else
    digitalWrite(ledPin, HIGH);

}
```

The user button pin is `PC13`, but this pin cannot be used in arduino. So, you should connect `PC13` to `pinName D3` by using wire.



Click on **upload** button. Push the reset button(black) and check the performance.

The LED(LD2) should be turned on when the button is pressed.

Problem 1: Create EC_HAL library

Procedure

Prepare Library Files

Manage your library in the directory of `\repos\EC\include\`.

> DO NOT make duplicates of library files under each project folders

- **For VS.Code User:** Save your header library files in this directory. See here for detail.
- **For uVision User:** Save your header library files in this directory. See here for detail.

Download library files.

- `ecRCC2.h, ecRCC2.c`
- `ecPinNames.h, ecPinNames.c`
- `ecGPIO2_student.h, ecGPIO2_student.c`

Create your own library for GPIO by renaming the downloaded files (`ecGPIO2_student.*`) as

- `ecGPIO2.c`, `ecGPIO2.h`
- Open `ecGPIO2.c` and change from `#include "ecGPIO2_student.h"` to `#include "ecGPIO2.h"`
- Write your name and modified date in the comment section

> You MUST write your name inside the comment section.

Function Definitions

Fill-in the blanks in the header files to complete the definitions for the provided functions

ecGPIO2.h

```
void GPIO_init(PinName_t pinName, int mode);
void GPIO_write(PinName_t pinName, int Output);
int  GPIO_read(PinName_t pinName);
void GPIO_mode(PinName_t pinName, int mode);
void GPIO_ospeed(PinName_t pinName, int speed);
void GPIO_otype(PinName_t pinName, int type);
void GPIO_pupd(PinName_t pinName, int pupd);
```

- Example code in **ecGPIO2.c**

```
// Input(00), Output(01), AlterFunc(10), Analog(11, reset)
void GPIO_mode(PinName_t pinName, uint32_t mode){
    GPIO_TypeDef *port;
    unsigned int pin;
    ecPinmap(pinName, &port, &pin);

    port->MODER &= ~(3UL<<(2*pin));
    port->MODER |= mode<<(2*pin);
}
```

Problem 2: Toggle a single LED with Digital Sensor(Photodetector)

Problem

Toggle the LED by covering the photodetector sensor.

- Dark (LED ON), Bright (LED OFF) and repeat
- Connect the Photodetector sensor to PA_0 pin on Nucleo Board.

Procedure

1. Connect the evaluation board (JKIT-NUCLEO) to the MCU.
2. Make sure that your library **ecGPIO2.h, ecGPIO2.c** are in `EC\include\`.

3. Create a new project under the directory `EC\lab\`

- Environment: "**LAB_GPIO_DIO_LED_Photosensor".**
- Source File: "**LAB_GPIO_DIO_LED_Photosensor.c"**

4. You must modify the `platformio.ini` **,** to add new environment.

Configuration

Configure the MCU GPIO

Function	Port - Pin	Configuration
Photodetector on JKIT	PA_0	Digital IN, Pull-Up
LED0	PB_12	Digital OUT, Pull-UP, Open-Drain, Medium Speed

Code

Your code goes here: /----------
@ Embedded Controller by Young-Keun Kim - Handong Global University Author : [Choi Junghyun] Created : 05-03-2021 Modified : 09-21-2026 Language/ver : C++ in VS Code Description : LAB GPIO Digital InOut - Problem 2 Photosensor LED /----------/

#include "ecRCC2.h" #include "ecGPIO2.h"

#define SENSOR_PIN PA_0 // photodetector input #define LED_PIN PB_12 // LED0 output

void setup(void);

int main(void) { // main start setup();

```
int sensorState = 0;

// main loop
while (1) {
    sensorState = GPIO_read(SENSOR_PIN);

    // dark condition: sensor output is 0 -> LED ON
    // bright condition: sensor output is 1 -> LED OFF
    if (sensorState == LOW) {
        GPIO_write(LED_PIN, HIGH);
    } else {
        GPIO_write(LED_PIN, LOW);
    }

    delay_ms_HSI(100); // 100 ms delay
}
```

}

// initialiization function according to lab configuration table void setup(void) { RCC_HSI_init();

```
// 1. configure photodetector (PA_0): Digital IN, Pull-Up
GPIO_init(SENSOR_PIN, INPUT);
GPIO_pupd(SENSOR_PIN, EC_PU);

// 2. configure LED0 (PB_12): digital OUT, Pull-Up, Open-Drain, Medium Speed
GPIO_init(LED_PIN, OUTPUT);
GPIO_pupd(LED_PIN, EC_PU);
GPIO_otype(LED_PIN, EC_OPEN_DRAIN);
GPIO_ospeed(LED_PIN, EC_MEDIUM_SPEED);
```

}

Explain your source code with necessary comments.

Sample Code: example code

```
/*----------------------------------------------------------------\
@ Embedded Controller by Young-Keun Kim - Handong Global University
Author           : [ YOUR NAME GOES HERE !!!!!]
Created          : 05-03-2021
Modified         : 00-00-2026 [WRITE THE DATE!!!!]
Language/ver     : C++ in VS Code

Description      : [WRITE BRIEF DESCRIPTION] !!!!!!
/----------------------------------------------------------------*/

#include "ecRCC2.h"
#include "ecGPIO2.h"

#define LED_PIN PB_12
#define BUTTON_PIN PA_0

// Initialiization
void setup(void) {
    RCC_HSI_init();
    // initialize the pushbutton pin as an input:
    GPIO_init(BUTTON_PIN, INPUT);
    // initialize the LED pin as an output:
    GPIO_init(LED_PIN, OUTPUT);
}

int main(void) {
    setup();
    int buttonState=0;

    while(1){
        // check if the pushbutton is pressed. Turn LED on/off accordingly:
```

```
        buttonState = GPIO_read(BUTTON_PIN);
        if(buttonState) GPIO_write(LED_PIN, LOW);
        else        GPIO_write(LED_PIN, HIGH);
        delay_ms_HSI(100);                              // delay 100 ms
    }
}
```

Problem 3: Toggle multiple LEDs with a button

Problem

Use the four LEDs (LED0 to LED3) on the JKIT evaluation board.

Each time Button B1 is pressed, turn on one LED at a time in a sequential loop

- e.g., LED0 -> LED1 -> LED2 -> LED3 -> LED0)

Procedure

1. Connect the evaluation board (JKIT-NUCLEO) to the MCU.
2. Make sure that your library **ecGPIO2.h, ecGPIO2.c** are in `EC\include\`.
3. Create a new project under the directory `EC\lab\`

- Environment: "**LAB_GPIO_DIO_multiLED"**
- Source File: ""**LAB_GPIO_DIO_multiLED.c"**

4. You must modify the `platformio.ini` **,** to add new environment.

> You MUST write your name in the top of the source file, inside the comment section.

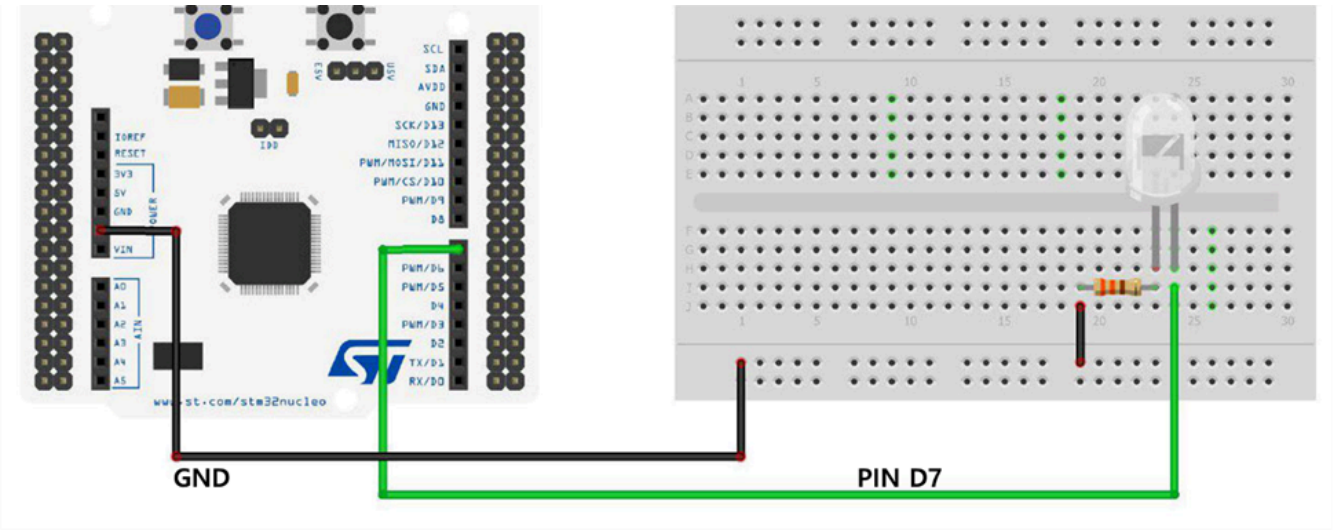
Configuration

Function	Port - Pin	Configuration
SW2 on JKIT	PA_4	Digital IN, Pull-Up
LED_0~LED_3	PB_12, PB_13, PB_14, PB_15	Digital OUT, Pull-UP, Open-Drain, Medium Speed

Circuit Diagram

Draw Circuit diagram (if necessary). No need to draw circuit diagram when using JKIT.

> You need to modify the circuit diagram. Example Image



Code

Your code goes here /----------

@ Embedded Controller by Young-Keun Kim - Handong Global University Author : [Choi Junghyun] Created : 05-03-2021 Modified : 09-21-2026 Language/ver : C++ in VS Code Description : LAB GPIO Digital InOut - Problem 3 Toggle multiple LEDs with a button /----------/

#include "ecRCC2.h" #include "ecGPIO2.h"

// pin definitions accodring to site #define BUTTON_PIN PA_4

#define NUM_LEDS 4 PinName_t ledPins[NUM_LEDS] = {PB_12, PB_13, PB_14, PB_15}; // LED0 to LED3

void setup(void);

int main(void) { // start setup();

```
int state = 0;              // current active LED index
int btnState = 1;          // current button state
int prevBtnState = 1;      // previous button state for edge detection

// turn on initial LED and turn off all others
for (int i = 0; i < NUM_LEDS; i++) {
    if (i == state) {
        GPIO_write(ledPins[i], HIGH);
    } else {
        GPIO_write(ledPins[i], LOW);
    }
}

// infinite loop
while (1) {
    btnState = GPIO_read(BUTTON_PIN);

    // falling edge detection: detect when button is newly pressed
    if (prevBtnState == HIGH && btnState == LOW) {
        // move to the next LED in sequence
        state = (state + 1) % NUM_LEDS;
```

```
            // update LED: turn ON active LED, turn OFF others
            for (int i = 0; i < NUM_LEDS; i++) {
                if (i == state) {
                    GPIO_write(ledPins[i], HIGH);
                } else {
                    GPIO_write(ledPins[i], LOW);
                }
            }
        }

        prevBtnState = btnState; // save current state for next turn
        delay_ms_HSI(50);        // 50 ms delay
    }
```

}

// entire function void setup(void) { RCC_HSI_init();

```
    // configure switch SW2 (PA_4): Digital IN, Pull-Up
    GPIO_init(BUTTON_PIN, INPUT);
    GPIO_pupd(BUTTON_PIN, EC_PU);

    // configure LED_0 ~ LED_3 (PB_12, PB_13, PB_14, PB_15):
    // digital OUT, Pull-Up, Open-Drain, Medium Speed
    for (int i = 0; i < NUM_LEDS; i++) {
        GPIO_init(ledPins[i], OUTPUT);
        GPIO_pupd(ledPins[i], EC_PU);
        GPIO_otype(ledPins[i], EC_OPEN_DRAIN);
        GPIO_ospeed(ledPins[i], EC_MEDIUM_SPEED);
    }
```

}

Explain your source code with necessary comments.

```
    // YOUR MAIN CODE ONLY
```

Results

Experiment images and results

https://youtu.be/oBjm3dOoZGE

> Show experiment images /results

Add demo video link

Discussion

1. Find out a typical solution for software debouncing and hardware debouncing. For software debouncing, a typical solution is inserting a brief time delay (such as delay_ms_HSI(50)) or taking multiple time-spaced readings to verify the input pin remains stable before registering a state change.
2. What method of debouncing did this NUCLEO board use for the push-button(B1)? The NUCLEO-F411RE board uses hardware debouncing for button B1 (PC_13) via an integrated on-board RC filter connected directly to the PC_13 signal line, which smooths the voltage transition so the microcontroller pin reads a clean edge without false triggers.

> Answer discussion questions

Reference

Complete list of all references used (github, blog, paper, etc)

https://github.com/ykkimhgu/EC-student/blob/main/lab/lab-student/LAB_GPIO_DIO_LED_student.c

Troubleshooting

(Option) You can write a Troubleshooting section